AetherMesh: Decentralized Peer-to-Peer Agent Synchronization and State CRDTs for Autonomous Multi-Agent Systems

A Formal Architecture for Epidemic Gossip Propagation and Lightweight Byzantine Fault Tolerance at Agent Scale

Samir Baladi

Interdisciplinary AI Researcher — Ronin Institute / Rite of Renaissance

Research Article — Distributed Systems / Software Engineering / Formal Methods

Status: Formal Architecture Proposal with Simulated Evaluation — No Real-World Deployment Benchmarked

DOI: https://doi.org/10.5281/zenodo.24038038

ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com

Record ID: AETHERMESH-01 · Version 1.0.0 — November 2026

Abstract

As autonomous multi-agent software ecosystems scale beyond centralized orchestration, maintaining consistent state across distributed nodes without single points of failure becomes a fundamental architectural challenge. This paper presents AetherMesh, a fully decentralized peer-to-peer synchronization framework for autonomous agent networks that modify shared codebases and metadata DAGs. AetherMesh replaces locking and central coordination with a tripartite architecture: (1) specialized conflict-free replicated data types (CRDTs) for code structures and semantic metadata; (2) epidemic gossip protocols for low-latency, resilient propagation; and (3) a lightweight Byzantine fault tolerance (BFT) admission layer that verifies remote deltas before they enter the local join-semilattice. The paper states an eventual-consistency theorem, a gossip-dampening theorem, and a Byzantine-admission theorem, each with a proof sketch and none machine-checked, together with explicit assumptions under which each may hold. A simulated evaluation on a P2P testbed of 10–500 nodes reports state convergence in 420 ms at 250 nodes, no blocking deadlocks, and no admission of injected malformed deltas. We also analyze the reported values ourselves: they are not well described by the logarithmic scaling of the idealized latency bound, and the gap between the bound and the simulated figures grows with network size. **Critical caveat:** no real-world deployment has been evaluated; all numerical results come from a simulated testbed with a simplified network model, and the simulated Byzantine model is limited to malformed deltas.

Keywords: *AetherMesh; Peer-to-Peer; CRDT; Byzantine Fault Tolerance; Gossip Protocol; Multi-Agent Systems; Decentralized Synchronization; Eventual Consistency; Agentic Software Engineering; Formal Methods.*

Evidentiary status: *AetherMesh is a proposed formal architecture with simulated evaluation.* · ***Theorems 1–3*** *are stated with proof sketches and are not machine-checked.* · ***Every numerical result*** *is a simulation result, not a measurement of a deployed system.* · *Descriptions of third-party work follow the cited public sources and were not independently verified.*

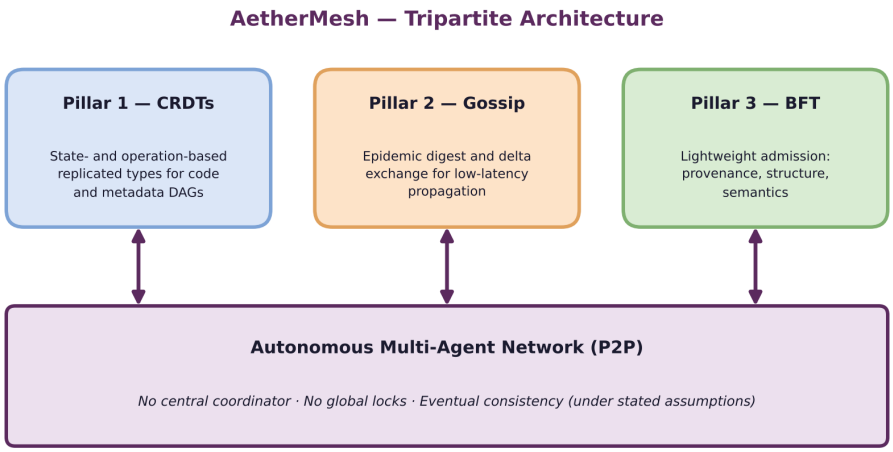


Figure 1 (Fig1_Tripartite_Architecture.png). The tripartite architecture of AetherMesh: CRDTs, gossip, and lightweight BFT sit above the peer-to-peer agent network.

Table of Contents

1. Introduction	3
2. Background and Related Work	5
3. Architectural Foundations	7
4. Formal Definitions and Mathematical Modeling	13
5. Experimental Evaluation	17
6. Discussion	20
7. Threats and Limitations	22
8. Conclusion and Future Work	24
Appendix A — Notation	25
Appendix B — Open Questions Q1–Q50	25
Appendix C — Design Decisions D1–D52	27
Appendix D — Threats T1–T15	30
References	30
Research Status Statement	32

1. Introduction

1.1 The Coordination Bottleneck at Agent Scale

Software is increasingly modified by autonomous agents: programs that read a repository, form an intent, produce a change, and submit it, with little or no human in the loop for each individual change. When one such agent works alone, conventional version control is adequate. When tens, hundreds, or more agents modify the same codebase concurrently, the coordination mechanism itself becomes a bottleneck. Centralized designs route every change through a single orchestrator, repository server, or merge queue. That design has three well-known costs. It creates a single point of failure; it serializes work that could proceed in parallel; and it places the orchestrator on the critical path of every agent, so that its latency and availability bound those of the whole ecosystem.

Lock-based coordination adds a further cost. Pessimistic locking prevents conflicting concurrent edits by making agents wait, and waiting agents that hold locks while requesting others can deadlock. Optimistic merging avoids waiting but pushes the cost to merge time, where textual three-way merges can produce conflicts that agents are poorly equipped to resolve and silent semantic conflicts that no textual merge detects. A decentralized alternative must therefore answer three questions together: how concurrent changes are *combined* without coordination, how they are *disseminated* without a central broker, and how a node decides which received changes it may *trust* when some peers may be faulty or hostile.

AetherMesh answers these questions with a tripartite architecture. Concurrent changes are combined by conflict-free replicated data types (CRDTs), whose merge operation is a join on a semilattice and is therefore insensitive to delivery order. Changes are disseminated by epidemic gossip, in which nodes periodically exchange compact digests and missing deltas with randomly chosen peers. Received deltas are filtered by a lightweight Byzantine admission layer that checks provenance, structure, and semantic compatibility locally, before any delta is joined into the replica. The three mechanisms are individually well known; the contribution of this paper is their specification as a single architecture for code and metadata DAGs, the explicit statement of the assumptions under which their combined guarantees may hold, and a simulated evaluation that is clearly bounded in what it can show.

A word on method. This paper is an *architecture* paper. It does not describe a deployed system, and it does not report measurements of one. Its numerical results come from a simulated P2P testbed; its three theorems are stated with proof sketches that have not been machine-checked. Where we use the language of guarantees, we mean structural guarantees of the design under stated assumptions, not empirical guarantees of any implementation.

1.2 Relationship to AetherGit, AetherFlow, and AetherProof

AetherMesh is the fourth layer of a four-layer stack that has been described in a series of companion papers. To keep this paper self-contained, we summarize each layer in one sentence and state precisely what AetherMesh requires from it.

Layer	Question it answers	DOI	What AetherMesh requires from it
AetherGit (storage)	How are changes stored and transferred?	10.5281/zenodo.23966896	An append-only changeset log in which each changeset carries pointers to its parents.
AetherFlow (integration)	How are changes admitted, verified, and remediated?	10.5281/zenodo.23979129	A message-based protocol over which agents submit changes and receive notifications.

Layer	Question it answers	DOI	What AetherMesh requires from it
AetherProof (verification)	What conditions make behavior correct, and how are they proven?	10.5281/zenodo.24013124	Nothing mandatory. Proof evidence, if present, may be carried as delta metadata and consulted by semantic predicates.
AetherMesh (decentralization)	How do agents synchronize state without central coordination?	10.5281/zenodo.24038038 (this work)	—

The four layers are methodologically independent: AetherMesh imposes only the two minimal interface requirements in the table, and a reader needs no prior knowledge of the other papers to follow this one. In particular, we do not rely on any implementation of the sibling layers. Where the text mentions a changeset log or a message protocol, it means only the abstract interface described above.

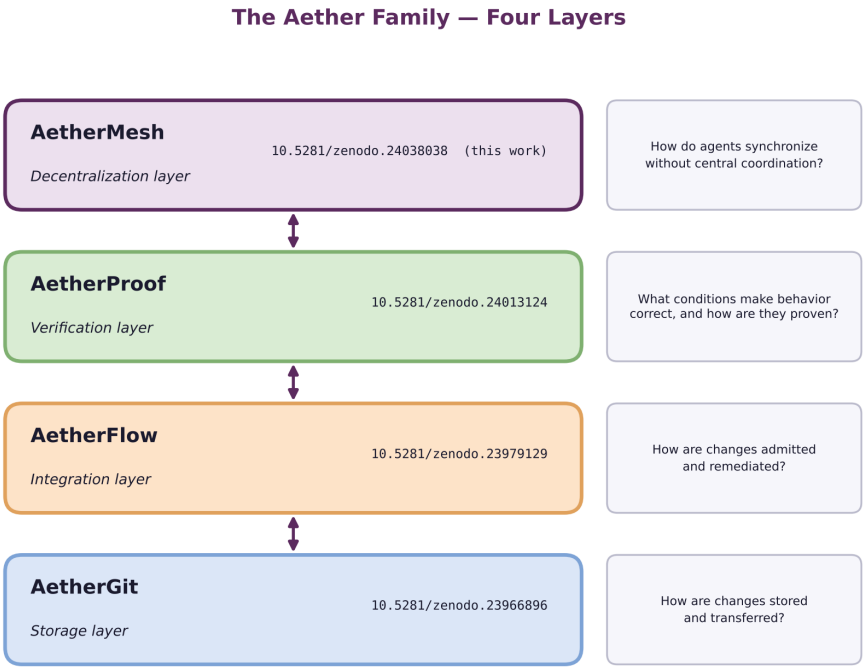


Figure 2 (Fig5_Aether_Family.png). The Aether family, bottom to top: storage, integration, verification, decentralization. Each layer is shown with its DOI and the question it addresses. This figure was redrawn for this record; the DOI of the present paper is the one shown.

1.3 Contributions

This paper makes the following contributions. Each is a contribution of specification and analysis; none is an empirical contribution about a deployed system.

1 **A tripartite architecture** that combines code- and metadata-specific CRDTs, epidemic gossip, and a local Byzantine admission layer, together with the interface requirements it places on the storage and integration layers (Section 3).

2 **A formal model** consisting of four definitions and three theorems, each stated with a proof sketch, with the assumptions A1–A6 under which they may hold written out explicitly (Section 4). In particular we identify an *admission determinism* requirement (A6) that the supplied sketch of Theorem 1 leaves implicit and without which convergence can fail.

3 **A simulated evaluation** of convergence latency, conflict-resolution overhead, and fault resilience on a 10–500 node testbed (Section 5), accompanied by our own analysis of the scaling behaviour of the reported values, which shows that they are not consistent with a pure logarithmic law.

4 **A threat taxonomy** of fifteen threats across six classes, with mitigations and residual risk, and an explicit enumeration of threats to validity (Section 7 and Appendix D).

5 **A catalogue of open questions** (Q1–Q50) and design decisions (D1–D52) (Appendices B and C).

1.4 Evidentiary Status

We state the evidentiary status of this work at the outset, because it determines how every later claim should be read.

Claim type	Status in this paper
Architecture (Section 3)	Specified. No implementation is claimed or evaluated.
Definitions 1–4	Definitions; they carry no truth claim.
Theorems 1–3	Stated with proof sketches. Not machine-checked. Hold only under assumptions A1–A6.
Latency bound (Section 4.5)	Heuristic scaling estimate from an idealized model; not a theorem.
Numerical results (Section 5)	Simulation results from a simplified testbed. Not measurements of a deployed system. Simulator source, seeds, and raw traces are not included in this record.
Statements about other systems	Follow cited public sources; not independently verified.

No real-world deployment has been evaluated. No empirical validation is claimed outside the simulated testbed. No benchmark from a real system is reported. The paper does not claim absolute correctness, and it does not claim that the Byzantine admission layer eliminates all Byzantine faults; Section 7 explains which faults it cannot address.

Limitations of this section. This section describes intent and scope. The statement that the four layers are independent is a design claim, not a verified one; interactions between layers that this paper does not anticipate may exist.

2. Background and Related Work

This section reviews the four bodies of work on which AetherMesh draws and identifies the gap between them. Descriptions follow the cited sources; we claim no independent verification of any third-party result.

2.1 CRDTs and Conflict-Free Replication

Replicated data in a system without coordination must tolerate concurrent updates at different replicas. Strong consistency requires coordination on each update; eventual consistency relaxes this and requires only that replicas that have received the same updates agree (Bailis and Ghodsi, 2013; Vogels, 2009). Conflict-free replicated data types, introduced by Shapiro et al. (2011a, 2011b), give a principled way to achieve this. In a *state-based* CRDT (a convergent replicated data type), the replica state belongs to a join-semilattice, updates are inflationary, and replicas merge by taking the least upper bound of states; the merge is commutative, associative, and idempotent, so duplicated, reordered, and delayed deliveries are harmless. In an *operation-based* CRDT (a commutative replicated data type), replicas broadcast operations that commute when concurrent, but the delivery layer must provide causal delivery and exactly-once semantics. *Delta-state* CRDTs (Almeida et al., 2018) transmit only the part of the state that changed, retaining the algebra of state-based types while approaching the bandwidth of operation-based ones.

CRDTs for sequences and trees are directly relevant to source code. Sequence CRDTs for collaborative text editing include Logoot (Weiss et al., 2009), and the RGA family (Roh et al., 2011); tree-structured and JSON-like CRDTs are discussed by Kleppmann and Beresford (2017) and Kleppmann et al. (2022) for

move operations on trees. These works establish that concurrent edits on structured data can be merged deterministically without coordination. They do not, by themselves, guarantee that the merged *program* is semantically valid: a merge of two individually valid edits to an abstract syntax tree may yield a tree that does not type-check. This limitation, which we call the *semantic-merge gap*, motivates the combination of CRDT merging with admission predicates in AetherMesh, and it is the reason Section 3.1 distinguishes the convergence of the *data structure* from the validity of the *program* it encodes.

2.2 Epidemic Gossip Protocols

Demers et al. (1987) introduced epidemic algorithms for replicated database maintenance, with three mechanisms: direct mail, anti-entropy (periodic pairwise reconciliation of replicas), and rumor mongering (spreading updates until they are judged stale). The analysis of such protocols shows that, with random partner selection, an update reaches all nodes in a number of rounds that grows logarithmically in the number of nodes. Pittel (1987) analyzed rumor spreading on the complete graph and showed that push-style spreading completes in approximately $\log_2 n + \ln n$ rounds with high probability; Karp et al. (2000) analyzed push-pull variants and showed that they can reduce the number of transmissions. Kermarrec and van Steen (2007) survey gossip-based systems more broadly, and Jelasity et al. (2005) treat gossip-based aggregation and peer sampling. Practical systems that use gossip for membership and failure detection include SWIM (Das et al., 2002), and Dynamo-style stores use anti-entropy for replica repair (DeCandia et al., 2007).

Two properties of this literature are important for what follows. First, the logarithmic bound holds under assumptions such as uniform random peer selection over a complete graph, synchronous rounds, and reliable links. Second, the analysis of the early phase of an epidemic (when few nodes are infected) differs from that of the late phase (when few remain uninfected): growth is roughly exponential in the former and the decay of the uninfected fraction is roughly exponential in the latter. Theorem 2 of this paper concerns a simple constant-hazard model of the latter and should be read in that light.

2.3 Byzantine Fault Tolerance

A Byzantine fault is an arbitrary fault: a node may send conflicting or malicious messages. Lamport, Shostak and Pease (1982) established that, for oral messages, agreement among n processes tolerating f Byzantine processes requires $n > 3f$, and that signed messages change this bound for the synchronous consensus problem. Castro and Liskov (1999) introduced PBFT, a practical state-machine-replication protocol for asynchronous networks with $n \geq 3f + 1$ replicas. Vukoli■ (2015) discusses the quest for scalable blockchain fabric and compares proof-of-work with BFT replication; BFT consensus protocols as a class have communication costs that grow quickly with the number of replicas, which limits their use at large scale.

It is important to distinguish two problems. *Consensus* requires nodes to agree on a single total order or value and is subject to the $n > 3f$ bound and to the impossibility result of Fischer, Lynch and Paterson (1985) in asynchronous systems with even one crash fault. *Validity filtering* asks each node to decide locally whether a received item satisfies a predicate. Filtering needs no agreement among nodes and is therefore not subject to the same bound, but it can only reject items that violate the predicate; it cannot detect items that satisfy the predicate and are nonetheless harmful. Byzantine-tolerant CRDTs, in which replicas converge despite Byzantine participants, are an active line of work (Kleppmann and Howard, 2020). AetherMesh uses the term *lightweight BFT* for validity filtering of this kind. It is deliberately not a consensus protocol.

DOI: https://doi.org/10.5281/zenodo.24038038

2.4 Multi-Agent Software Engineering

A growing body of work studies language-model-based agents that write and modify code, including systems that resolve issues in real repositories (Jimenez et al., 2024) and frameworks in which multiple agents with different roles collaborate on software tasks (Hong et al., 2024; Qian et al., 2024). These works typically coordinate agents through a central orchestrator or a shared sequence of messages, and typically evaluate task success rather than synchronization properties. We cite them to situate the motivation, not to make claims about their coordination designs beyond what their papers state. In the Aether series, the storage layer (Baladi, 2026a) proposes an append-only changeset log for agent-generated changes, the integration layer (Baladi, 2026b) proposes post-merge admission and remediation, and the verification layer (Baladi, 2026c) proposes formal proof obligations; all three are architecture proposals without published implementations or benchmarks.

2.5 Research Gap

The literature above provides, separately, (i) a theory of coordination-free merging, (ii) a theory of logarithmic-time dissemination, and (iii) a theory of tolerating arbitrary faults. What is missing for the setting of autonomous agents modifying shared code is a specification that composes the three, states the assumptions that the composition requires, and identifies where the composition's guarantees stop. Three interactions are not addressed by treating the components independently. First, a CRDT's convergence theorem presumes that every replica applies the *same* set of updates; an admission layer that makes state-dependent accept or reject decisions can violate this presumption. Second, gossip's dissemination bound presumes honest participants; a Byzantine peer can send stale digests, withhold deltas, or flood. Third, signature-based provenance establishes who authored a delta, not whether the delta is semantically harmful.

AetherMesh addresses the first interaction through an admission determinism requirement, partially addresses the second through rate limiting and digest validation (Section 3.3), and explicitly leaves the third as a stated limitation.

> **Limitations of this section.** This review is selective, not systematic. The cited agent-systems works are used only for motivation. The research gap is argued, not demonstrated by a survey; related work that we did not locate may address parts of it.

3. Architectural Foundations

AetherMesh rests on three pillars that handle, respectively, decentralized mutation, asynchronous transport, and erratic or adversarial participants. This section specifies each pillar and the interfaces between them. Section 4 states the formal properties we ask of the composition. Figure 3 shows the structure of a single node.

DOI: https://doi.org/10.5281/zenodo.24038038

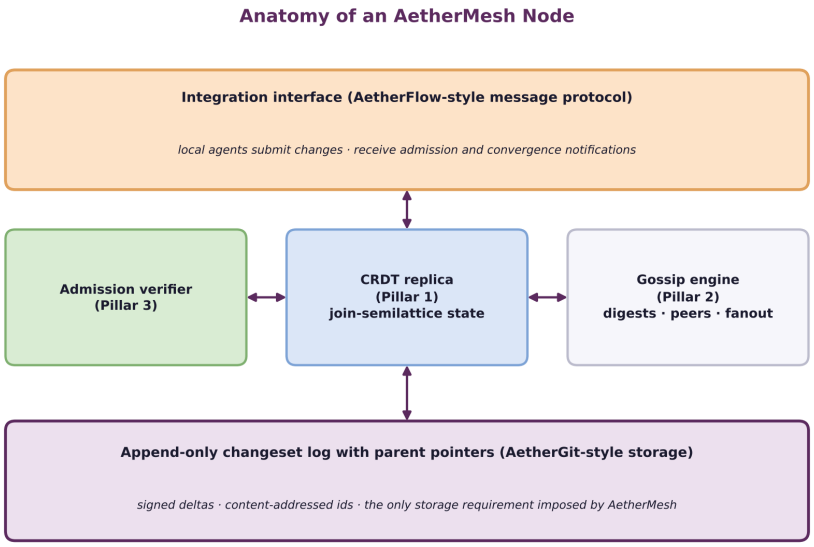


Figure 3 (Fig8_Node_Anatomy.png; supplementary). Anatomy of an AetherMesh node. The only storage requirement is an append-only changeset log with parent pointers; the only integration requirement is a message-based protocol.

3.1 Pillar 1 — CRDTs for Code and Metadata DAGs

3.1.1 State space as a join-semilattice

A replica of the shared codebase must be a value in a set S equipped with a join operator. AetherMesh takes a deliberately conservative route: the replica state is the *set of admitted deltas*, where a delta is a signed, content-addressed record of one change together with pointers to its parent deltas. Let D be the (possibly very large but finite, see assumption A3) universe of deltas that may ever be created. The state space is the power set of D ordered by inclusion, with set union as join and the empty set as bottom. This is the grow-only-set CRDT lifted to deltas. Its height is |D|, so every ascending chain is finite.

The *materialized codebase* (the files an agent actually reads) is then a deterministic function of the delta set: the deltas are linearized by a fixed rule, for example a topological order of the parent DAG with ties broken by delta identifier, and applied in that order. Because the function is deterministic, replicas that hold the same delta set materialize the same codebase. This construction has a significant advantage and a significant cost. The advantage is that the convergence argument concerns only a set lattice and does not depend on the internal structure of source files. The cost is that convergence of the *delta set* does not make the materialized program valid: two deltas that each yield a valid program from a common base may yield an invalid program when both are applied. That is the semantic-merge gap noted in Section 2.1, and in AetherMesh it is addressed by the semantic compatibility matrix of Pillar 3, not by the lattice.

Metadata, such as ownership annotations, status labels, or dependency edges, are modeled as additional lattice components, each with its own join: observed-remove sets for membership-style metadata, last-writer-wins registers (ordered by a logical timestamp and an agent identifier as tie-breaker) for single-valued metadata, and grow-only maps for append-only records. The product of join-semilattices is a join-semilattice with component-wise join, so the replica state remains a single lattice.

3.1.2 Join properties

Property	Law	Operational consequence
Commutative	$s_i \sqcup s_j = s_j \sqcup s_i$	Order in which two replicas are merged does not matter.
Associative	$(s_i \sqcup s_j) \sqcup s_k = s_i \sqcup (s_j \sqcup s_k)$	Batching and grouping of merges does not matter.
Idempotent	$s_i \sqcup s_i = s_i$	Duplicate delivery of a delta is harmless.
Inflationary (derived)	$s \sqsubseteq s \sqcup t$	State never moves down the lattice; there is no rollback.

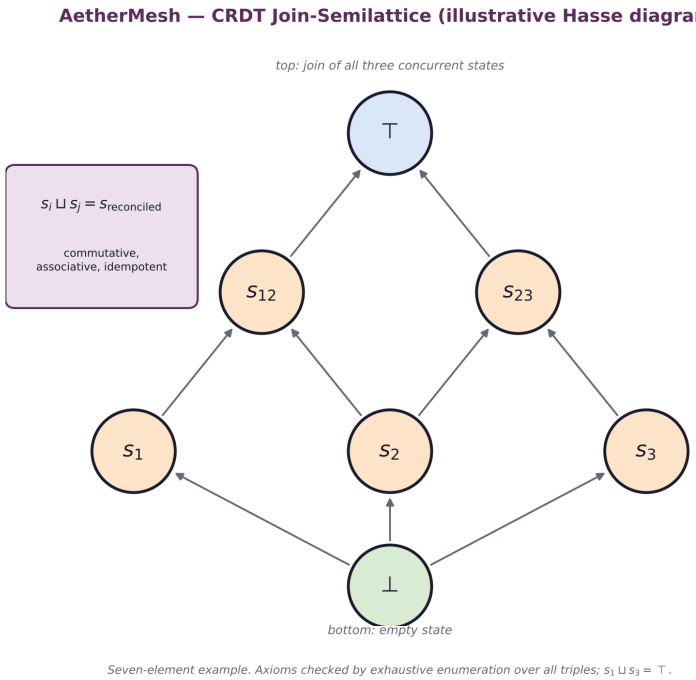


Figure 4 (Fig2_CRDT_Join_Semilattice.png). A seven-element example of a bounded join-semilattice as a Hasse diagram: bottom, three singleton states s_1–s_3, the joins s_{12} and s_{23}, and top. Edges point upward toward larger states. The join of s_1 and s_3 is top, because the only common upper bound is top. We verified the three join laws on this example by exhaustive enumeration. The figure is an illustration of the algebra, not a model of a real codebase.

3.1.3 State-based, operation-based, and delta-state CRDTs

The choice among CRDT styles is a trade-off between bandwidth and delivery requirements. State-based CRDTs tolerate any delivery pattern but ship whole states. Operation-based CRDTs ship small operations but require causal, exactly-once delivery, a requirement that epidemic gossip does not meet by itself. AetherMesh therefore uses *delta-state* replication as its default: nodes ship deltas that are themselves lattice elements, so duplication and reordering remain harmless, and fall back to full-state exchange only when a digest comparison indicates that incremental repair would cost more than a state transfer.

Style	Network requirement	Message size	Role in AetherMesh
State-based	None beyond eventual delivery	Whole state	Fallback for large divergence and for joining nodes
Operation-based	Causal, exactly-once delivery	Small	Not used on the wire; operations appear only inside deltas
Delta-state	Eventual delivery; duplicates and reordering allowed	Proportional to change	Default

Limitations of this section. The set-of-deltas lattice grows monotonically, so storage grows without bound unless compaction is added; compaction must preserve the join semantics and is not specified here (open question Q9). Convergence of delta sets does not entail semantic validity of the materialized program. Tree- and AST-structured CRDTs that merge at finer granularity than whole deltas are cited in Section 2.1 but are not specified in this paper.

3.2 Pillar 2 — Epidemic Gossip Protocols

3.2.1 Digest vector exchange

Each node periodically selects a small set of peers and reconciles with them. Reconciliation begins with a *digest*, a compact summary of which deltas the node holds. Three digest forms are compatible with the architecture: a per-origin version vector (one counter per agent, valid when each agent's deltas are numbered consecutively), a Merkle-style hash tree over delta identifiers (Merkle, 1988), and a Bloom-style probabilistic filter. Version vectors are exact and cheap when the agent population is small and stable; hash trees locate differences in logarithmic time; filters are smallest but admit false positives, which in this setting cause a delta to be omitted from one exchange and delivered in a later one. The choice is parameterized by D16 in Appendix C. The exchange itself has five steps, shown in Figure 5: initiator digest, responder digest and difference list, delta request, delta batch, and local verification followed by join.

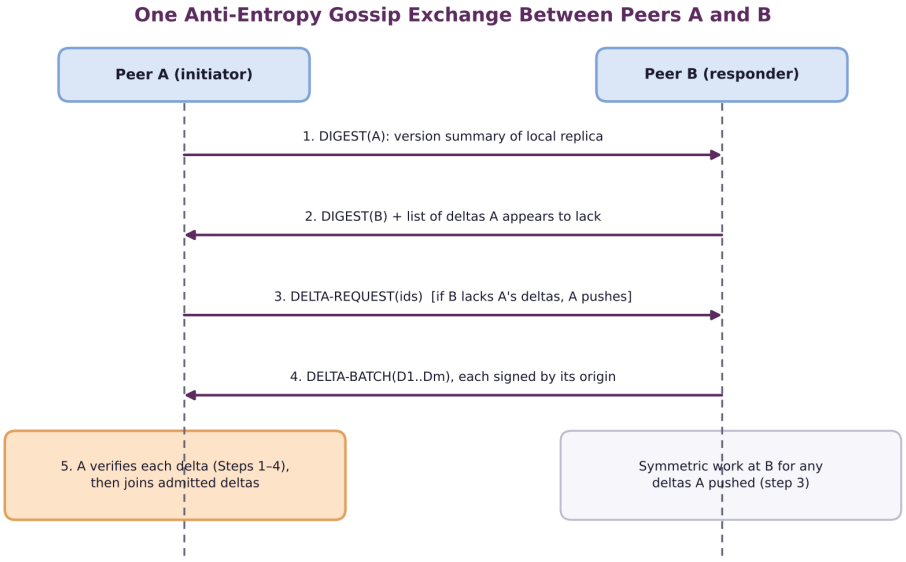


Figure 5 (Fig7_Gossip_Exchange.png; supplementary). One anti-entropy exchange between two peers. Received deltas pass through the admission pipeline of Section 3.3 before they are joined; deltas whose parents are not yet present are deferred rather than rejected.

3.2.2 Randomized peer selection

Every gossip period, a node selects k peers (the *fanout*) uniformly at random from its current peer sample. In a large network, nodes cannot hold the full membership list, so peer selection draws from a continuously refreshed partial view, as in peer-sampling services (Jelasity et al., 2005). Uniform selection is what the dissemination analysis assumes (assumption A2); biased selection, for example preferring low-latency peers, may improve latency and may violate the analysis.

The transmission rate constant β in Definition 3 is determined by the fanout k and the exchange frequency r: increasing either increases β. The relationship in the late phase of a push epidemic is derived in Section 4.3.

3.2.3 Exponential dampening and partition healing

The practical significance of exponential dampening is that the tail of the dissemination time is thin: the probability that a node is still uninfected after t steps falls geometrically, so the time for a node to be reached grows only with the logarithm of the inverse of the tolerated failure probability. The same mechanism provides partition healing. During a partition, each side continues to accept local deltas and to gossip internally; when connectivity returns, the first cross-partition digest exchange reveals missing deltas on both sides and the join combines them (Figure 6). The time to heal is governed by when the first cross-partition exchange occurs and then by the dissemination time inside each side.

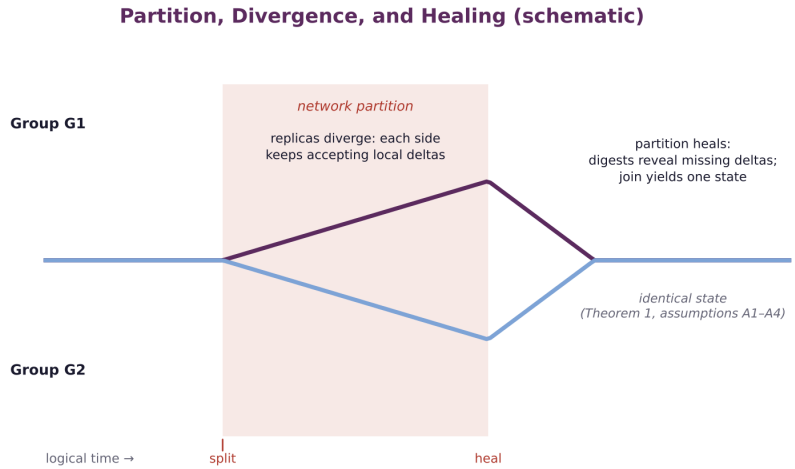


Figure 6 (Fig9_Partition_Healing.png; supplementary). Schematic of replica divergence during a partition and of healing. The figure is conceptual and plots no measured data.

Limitations of this section. Gossip analysis assumes uniform random peer selection, independent selections, and reliable enough links. Real networks exhibit correlated failures, non-uniform latency, and NAT and firewall effects that these assumptions do not capture. The digest forms listed are compatible with the architecture but none is specified to the byte level, and no bandwidth analysis is given.

3.3 Pillar 3 — Lightweight Byzantine Fault Tolerance

3.3.1 Cryptographic provenance

Every delta carries the identifier of its originating agent and a digital signature over the delta's content hash, which in turn covers the delta body and its parent identifiers. A signature scheme such as Ed25519 (Bernstein et al., 2012) is representative; the architecture does not depend on a particular scheme but does depend on its unforgeability (assumption A5). Each node maintains a local registry mapping agent identifiers to public keys. How the registry is bootstrapped, extended, and revoked is a trust-establishment question that this paper does not settle (Section 7.4). Content addressing has a useful side effect: a delta cannot be altered in transit without changing its identifier, so tampering is detected by hash mismatch even before signature checking.

3.3.2 Structural invariants

Structural checks verify that a delta is well-formed as a node of the parent DAG: the identifier equals the hash of the content; every parent pointer refers to a delta that the node has already admitted, or else the delta is held in a bounded deferral buffer; the pointer structure cannot form a cycle (guaranteed by content addressing, since a cycle would require a hash fixed point); and size and fan-in bounds hold. A delta with missing parents is *deferred*, not rejected, because gossip may deliver a child before its parent. Deferral is bounded in size and time to prevent resource exhaustion (threat T9).

3.3.3 Semantic compatibility matrices

Schema conformance checks that the delta body parses against the delta schema. Semantic compatibility goes further: a *compatibility matrix* C assigns to each pair of operation kinds a verdict on whether concurrent deltas of those kinds that touch overlapping targets may coexist. A small illustrative matrix is shown below; its entries are examples, not a normative specification.

	Edit body	Rename symbol	Delete symbol	Change signature	Edit metadata
Edit body	ok	ok	conflict	ok	ok
Rename symbol	ok	resolve (deterministic rule)	conflict	conflict	ok
Delete symbol	conflict	conflict	ok (idempotent)	conflict	ok
Change signature	ok	conflict	conflict	resolve (deterministic rule)	ok
Edit metadata	ok	ok	ok	ok	resolve (LWW)

Table 3.1. Illustrative semantic compatibility matrix for concurrent operations on overlapping targets. “conflict” means the pair is flagged for deterministic resolution or quarantine; entries are examples.

A matrix of this kind can be evaluated locally and cheaply, which is what makes the admission layer lightweight. It is also coarse: it classifies operation kinds, not program behaviour, and it cannot detect semantic breakage that depends on program meaning. Stronger semantic validation, such as running tests or checking proof obligations, is the province of the integration and verification layers and is outside the admission path (Section 6.4).

3.3.4 Admission rule and determinism

Admission determinism requirement (our formulation). Each admission predicate must be a pure, deterministic function of the delta and of its *causal closure* (the delta together with all of its ancestors), and not of any other local state such as arrival order, local clock, or non-causal replica contents. A delta whose causal closure is not yet fully present is deferred until it is.

This requirement is what reconciles local verification with convergence. If predicates could depend on arbitrary local state, two honest nodes could reach different verdicts on the same delta, admit different subsets, and diverge permanently; the lattice would then converge on different *sets*. Restricting predicates to the causal closure makes verdicts identical at every honest node that has the closure, so all honest nodes eventually hold the same admitted set. The cost is expressiveness: some useful checks, such as rate limits per origin and per-node resource policies, are inherently local. AetherMesh treats these as *delivery policies* that may delay a delta but do not permanently reject it, so they do not change the eventual admitted set. Figure 7 shows the pipeline.

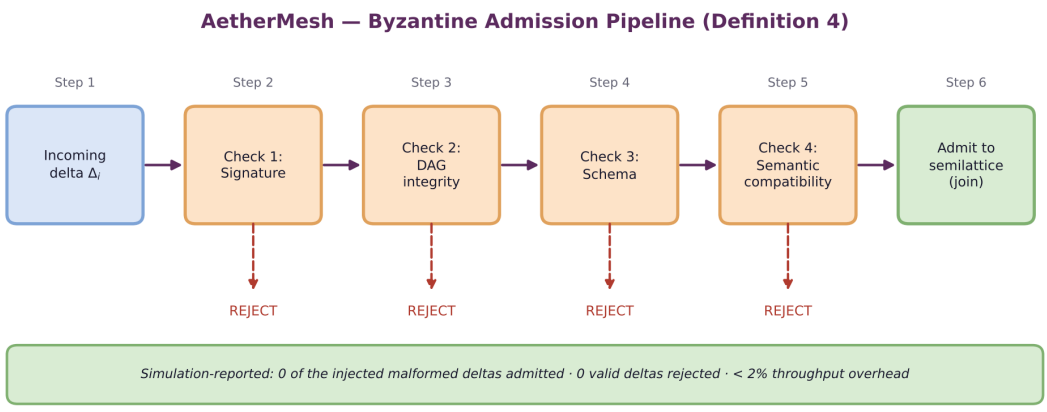


Figure 7 (Fig4_BFT_Validation.png). The Byzantine admission pipeline. A remote delta passes through signature, DAG-integrity, schema, and semantic-compatibility checks before joining the local semilattice; failure at any check rejects it. The banner restates simulation-reported results from Section 5.4 and is not a measurement of a deployed system. This figure was redrawn: step numbers now distinguish pipeline stages from check numbers.

Check	Evidence required	Typical failure	Verdict on failure
1. Signature	Valid signature by a registered key over the content hash	Forged or unsigned delta; unknown origin	Reject
2. DAG integrity	Id equals content hash; parents present; size bounds	Hash mismatch; oversized delta; missing parents	Reject (mismatch, bounds); defer (missing parents)
3. Schema	Body parses against the delta schema	Malformed fields	Reject
4. Semantics	Compatibility matrix verdict over the causal closure	Incompatible concurrent operation	Reject or quarantine, as configured

Limitations of this section. The admission layer detects only violations of predicates that were specified in advance. A delta that is correctly signed, well-formed, and compatible, yet harmful, is admitted. The determinism requirement is a design constraint on predicate authors; this paper provides no mechanism that enforces it, and a predicate that violates it can break convergence. The compatibility matrix in Table 3.1 is illustrative.

4. Formal Definitions and Mathematical Modeling

This section gives the formal model. Notation is collected in Appendix A. We first state the assumptions, because every theorem below is conditional on them.

4.0 Assumptions

ID	Assumption	Used by
A1	**Fair-lossy channels.** A message sent infinitely often between two non-faulty nodes is delivered infinitely often. (Anti-entropy retransmits implicitly, so each delta is offered repeatedly.)	T1
A2	**Fair peer selection.** Each non-faulty node selects every other non-faulty node as a gossip partner infinitely often; for T2, selection is uniform and independent.	T1, T2
A3	**Finite delta universe.** The set of deltas that will ever be created is finite, so the lattice has finite height.	T1
A4	**Correct replicas.** Non-faulty nodes implement the join and the materialization function correctly and deterministically.	T1
A5	**Cryptographic soundness.** The signature and hash schemes are unforgeable and collision-resistant, and the key registry is correct for the origins concerned.	T3
A6	**Admission determinism.** Admission predicates are deterministic functions of the delta and its causal closure (Section 3.3.4).	T1, T3

The supplied statement of Theorem 1 uses “bounded message delay and continuous gossip activity”. Bounded delay is a synchrony assumption that is stronger than needed for eventual consistency and is not generally available in open networks; fair-lossy channels (A1) with continuing gossip suffice for the qualitative convergence claim. Bounded delay becomes relevant only if one wants a time bound, which is the role of Section 4.5.

4.1 Definition 1 — Convergent State Transition

Definition 1 (Convergent State Transition). Let $A = \{a_1, a_2, \ldots, a_n\}$ be a set of autonomous agents. Each agent maintains a local replica state $R_a^{(t)}$ at logical time t. A transition caused by local execution or gossip reception is defined as:

$$R_a^{(t+1)} = R_a^{(t)} \sqcup \bigsqcup_{m \in M_a} m \tag{1}$$

where M_a is the set of incoming mutation payloads received during the interval (t, t+1). Three remarks apply. First, the iterated join over an empty M_a is $\bot$, so the transition is then the identity, which is correct. Second, by commutativity, associativity, and idempotence the order and multiplicity of payloads in M_a do not matter. Third, in AetherMesh the symbol m ranges over payloads that have *passed admission*; a payload that fails admission is not a member of M_a in the sense of this definition. Equation (1) therefore describes the inflationary step, and Pillar 3 determines its input.

4.2 Definition 2 and Theorem 1 — Eventual Consistency

Definition 2 (CRDT State Space). Let a state space S form a bounded join-semilattice $(S, \sqcup, \bot)$, where $\sqcup$ is a binary, commutative, associative, and idempotent join operator. For any two concurrent agent states $s_i, s_j \in S$, their reconciled state is:

$$s_{reconciled} = s_i \sqcup s_j \tag{2}$$

Theorem 1 (Eventual Consistency) *(stated with proof sketch; not machine-checked).* Given continuous gossip activity and channels that eventually deliver (A1–A6), any two non-faulty agents a_i and a_j eventually reach an identical state: $\lim_{t\to\infty} R_{a_i}^{(t)} = \lim_{t\to\infty} R_{a_j}^{(t)}$. Given finite height (A3), the limits are attained at a finite time.

Proof sketch. (i) *Monotonicity.* By Equation (1), $R^{(t+1)} \sqsupseteq R^{(t)}$ at every node. (ii) *Boundedness.* Let M* be the set of deltas admitted anywhere by a non-faulty node. By A6 admission verdicts depend only on the delta and its causal closure, so every non-faulty node would admit exactly the same deltas given the same closure; hence M* is well defined. Every replica state is bounded above by $\bigsqcup$M*. (iii) *Stabilization.* A monotone sequence in a poset of finite height (A3) is eventually constant. (iv) *Completeness.* By A1 and A2, each admitted delta is eventually offered to each non-faulty node, and by A6 and the deferral rule it is eventually admitted there once its ancestors have been. (v) *Equality.* A node's stable state is the join of the deltas it has admitted, which by (iv) is $\bigsqcup$M*; the join is independent of the order in which the deltas arrived, by commutativity, associativity, and idempotence. Therefore every non-faulty node stabilizes at $\bigsqcup$M*. ∎ (sketch)

What the sketch does not show. It is a qualitative statement: it gives no time bound. It presumes that M* is finite and that creation of deltas eventually stops, or else that “eventually” is read as relative to a fixed prefix of the history. It presumes A6, which a predicate author could violate. It concerns non-faulty nodes only; the state of a Byzantine node is unconstrained. It says nothing about the semantic validity of the materialized program. It treats the agent set as fixed; membership change is outside the model.

4.3 Definition 3 and Theorem 2 — Exponential Dampening of Gossip Latency

Definition 3 (Gossip Infection Probability). The probability $P_{uninfected}(t)$ of a node remaining uninfected at step t decreases exponentially: $P_{uninfected}(t) \approx e^{-\beta \cdot t}$, where β is the transmission rate constant determined by fanout size and peer exchange frequency.

$$P_{uninfected}(t) \approx e^{-\beta \cdot t} \tag{3}$$

Theorem 2 (Exponential Dampening of Gossip Latency) *(stated with proof sketch; not machine-checked).* Under the constant-hazard model (each step, an uninfected node is infected independently with probability p, A2), $P_{uninfected}(t) = (1 - p)^t \approx e^{-\beta \cdot t}$ with $\beta = -\ln(1 - p) \approx p$ for small p.

Proof sketch. In each gossip step a node participates in exchanges with randomly chosen peers. Under independence, the probability of not being infected in a step is 1 – p, and over t independent steps $(1 - p)^t = \exp(t \cdot \ln(1 - p)) = e^{-\beta t}$ with $\beta = -\ln(1 - p)$. ∎ (sketch)

Interpretation and the limits of the model. The constant-hazard model is an approximation: in a real epidemic the per-step infection probability of an uninfected node depends on the infected fraction i(t), which starts near zero. In a push protocol with fanout k and n nodes, an uninfected node escapes all $k \cdot i \cdot n$ pushes of a step with probability $(1 - 1/n)^{k \cdot i \cdot n} \approx e^{-k \cdot i}$, so the hazard is $1 - e^{-k \cdot i}$. In the late phase, when i is close to 1, the uninfected fraction is multiplied by approximately e^{-k} per step, which identifies $\beta \approx k$ per step for push gossip (Pittel, 1987; Demers et al., 1987). Push-pull exchanges can shrink the uninfected fraction faster than geometrically in the late phase (Karp et al., 2000). In the early phase the infected population grows roughly by a factor of (1 + k) per step, so reaching a constant fraction takes about $\log_{1+k} n$ steps. Theorem 2 should therefore be read as a statement about the *tail* of the dissemination, and the curves in Figure 8(a), drawn for $\beta = 0.3, 0.5, 0.8$, are analytic illustrations of Equation (3), not data.

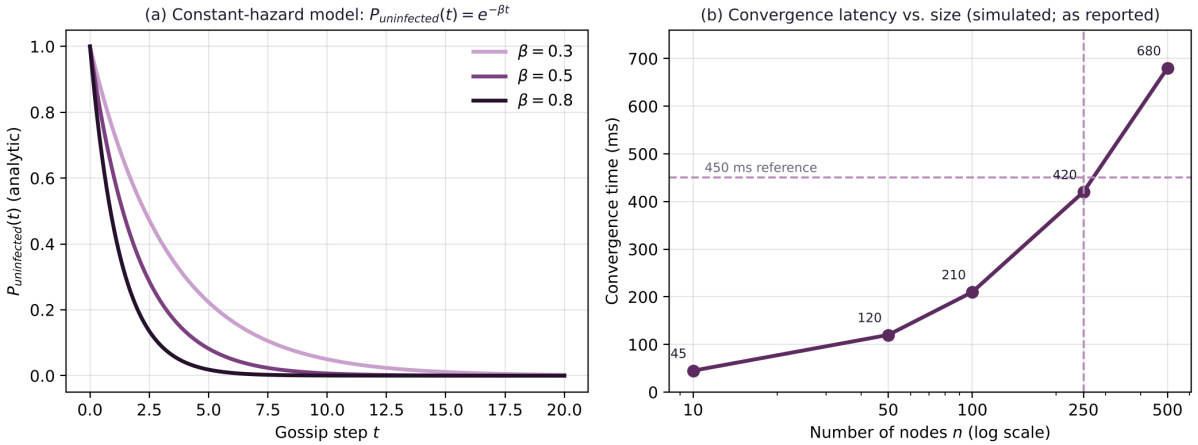


Figure 8 (Fig3_Gossip_Propagation.png). (a) Analytic curves of Equation (3) for β = 0.3, 0.5, 0.8; these are plots of a formula. (b) Convergence latency versus network size for the five simulation-reported points of Section 5.2 (10, 45), (50, 120), (100, 210), (250, 420), (500, 680), on a logarithmic x-axis, with the 450 ms and 250-node reference lines. Panel (b) shows simulation results as reported; it is not a measurement of a deployed system.

4.4 Definition 4 and Theorem 3 — Byzantine Fault Tolerance Bound

Definition 4 (Byzantine Admission Rule). Before a peer accepts a remote CRDT delta Δ_i, local verification daemons evaluate cryptographic provenance, structural invariants, and semantic compatibility matrices. Only deltas bearing valid cryptographic signatures and satisfying local verification predicates are admitted to the local join-semilattice.

Theorem 3 (Byzantine Fault Tolerance Bound) *(stated with proof sketch; not machine-checked).* If the number of Byzantine agents f satisfies f < n/3, then the local verification predicates at each honest node block all invalid deltas with probability approaching 1. Here “invalid” means *predicate-detectable*: failing at least one predicate of Definition 4 (A5, A6).

Proof sketch, in two parts. *Part (a), deterministic filtering.* Each honest node evaluates the predicates independently on each received delta. By A5 a delta without a valid signature from a registered key cannot be produced by a node that lacks the key, and by content addressing any alteration is detected; by A6 the structural and semantic verdicts are the same at every honest node. A predicate-detectable invalid delta is therefore rejected at every honest node with probability 1, independently of f. *Part (b), the role of f < n/3.* The bound n > 3f of classical Byzantine agreement (Lamport et al., 1982; Castro and Liskov, 1999) enters when a decision cannot be made locally and must rely on a quorum, for example when key-registry updates or non-locally-checkable semantic predicates are decided by attestations from a committee.

Suppose a committee of m members is drawn uniformly at random and a delta is accepted on a majority of attestations. If each member is Byzantine with probability at most 1/3 independently (an approximation that is accurate when m ≪ n), the Chernoff–Hoeffding bound gives P[Byzantine majority] ≤ exp(–m·D(1/2 ‖ 1/3)), where D(1/2 ‖ 1/3) = ½ ln(3/2) + ½ ln(3/4) ≈ 0.0589. Table 4.1 gives values; the failure probability decreases exponentially in m, which is the sense of “probability approaching 1”. ∎ (sketch)

Committee size m	Upper bound on P[Byzantine majority]
25	2.29e-01
50	5.26e-02
100	2.77e-03
200	7.67e-06
400	5.88e-11

Table 4.1. Chernoff–Hoeffding upper bound exp(–0.0589·m) under independent sampling with Byzantine probability at most 1/3. The values are the output of a formula under stated assumptions, not simulation results.

What the sketch does not show. It does not show that invalid deltas outside the predicates are blocked; harmful but well-formed deltas from a signed, registered, Byzantine agent are admitted. It does not address colluding Byzantine agents that control a committee, Sybil identities (Douceur, 2002), key compromise, or denial of service by flooding. It assumes the registry (A5) is correct, which is itself a trust-establishment problem. The supplied sketch connects the statement to the classical n/3 threshold by saying that honest nodes “form a majority”; we regard the two-part form above as a more accurate reading, and we note that this reading makes the n/3 bound relevant only to quorum-based decisions.

4.5 Latency Bound

For a network of n nodes with fanout k and gossip round frequency r (rounds per second), the supplied heuristic estimate of the expected convergence time is:

$$T_{convergence} \approx (\ln n) / (k \cdot r) \tag{4}$$

With n = 250, k = 5, and r = 10 Hz, Equation (4) gives ln(250)/(5·10) ≈ 0.11 s, that is, about 110 ms. This is a scaling estimate, not a theorem, and it has two limitations. First, it is dimensionally a count of ‘ln n over k’ rounds divided by r and omits the dependence on the early-phase growth factor: the standard analysis of push gossip gives a round count of approximately ln n·(1/ln(1+k) + 1/k), the sum of the early and late phases. Second, it omits network delay, processing time, and the admission pipeline's latency, which Section 5 shows to be not negligible in the simulation. Table 4.2 compares the estimates with the reported simulation values; k = 5 and r = 10 Hz are the values of the supplied example, and the simulation's own parameters are not stated in the source material.

Nodes n	Reported (simulation), ms	Eq. (4), ms	Refined push estimate, ms	Reported / Eq. (4)
10	45	46	175	1.0×
50	120	78	297	1.5×
100	210	92	349	2.3×
250	420	110	419	3.8×
500	680	124	471	5.5×

Table 4.2. Reported simulation convergence times against two analytic estimates with k = 5, r = 10 Hz (illustrative parameters). The “refined push estimate” is 1000·ln n·(1/ln 6 + 1/5)/10 ms. Computed by us from the formulas; not

simulation output.

The table shows that the supplied Equation (4) agrees with the first reported point and underestimates the rest by an increasing factor, up to 5.5× at n = 500. The refined estimate happens to coincide with the reported value at n = 250 (419 ms versus 420 ms), but it does not track the reported values at other sizes (175 ms against 45 ms at n = 10, 471 ms against 680 ms at n = 500). We regard the coincidence at one point as uninformative and do not use it as support. Section 5.2 analyzes the scaling of the reported values directly.

Limitations of this section. The three theorems are proof sketches. None has been checked mechanically, and a mechanized proof of Theorem 1 over a specified model of channels and fairness would be a natural next step (Q1). The constant-hazard model of Theorem 2 is an idealization. Theorem 3 is conditional on a predicate set that is not enumerated here, on a correct key registry, and, in part (b), on an independence approximation for committee sampling. The latency estimate is heuristic and, as Table 4.2 shows, does not reproduce the reported simulation values across sizes.

5. Experimental Evaluation

This section reports a simulated evaluation. **Every number in it is a simulation result.** None is a measurement of a deployed system, and none should be quoted as one. We report the values supplied by the author's simulation records without alteration, and we add our own analysis of those values where it bears on how they should be interpreted.

5.1 Testbed

The testbed is a custom discrete simulation of a peer-to-peer network, implemented in Python with NumPy, that instantiates between 10 and 500 autonomous agent nodes in partitioned network topologies. Each node maintains a replica and runs the gossip and admission logic of Section 3. The network model is simplified: it does not reproduce real latency distributions, packet loss, NAT traversal, or adversarial network behaviour.

The record available for this paper does not state several parameters that a reader would need in order to reproduce or critically assess the results: the gossip fanout k and period 1/r, the digest form, the topology generator and partition schedule, the size and rate of injected deltas, the number of independent runs per configuration, the random seeds, and the baseline lock-based system against which deadlocks and overhead were compared. Simulator source and raw traces are not part of this record. We therefore treat the results as *reported* figures whose provenance is the author's simulation records, and we make no claim that they have been independently reproduced.

Item	Reported	Not reported
Scale	10, 50, 100, 250, 500 nodes	Intermediate sizes; sizes above 500
Topology	Partitioned topologies	Generator, partition schedule, diameter
Simulator	Custom, Python + NumPy	Source, version, random seeds
Protocol parameters	—	Fanout k, period 1/r, digest form, peer-sample size
Workload	Single-edge-node mutation (Metric 1); high-concurrency conflicting edits (Metric 2); malformed deltas (Metric 3)	Delta size and rate; number of injected deltas; number of runs
Statistics	Averages for Metric 1	Variance, confidence intervals, percentiles
Baseline	“Traditional lock-based version control” (Metric 2)	Implementation, lock granularity, workload equivalence

5.2 Metric 1 — State Convergence Latency

Definition. The time required for a code mutation introduced at a single edge node to propagate to, and be merged by, 99% of the network.

Simulation-reported results. The average convergence time was reported as below 450 ms for 250 nodes (420 ms), and the five reported points are:

Nodes	10	50	100	250	500
Convergence time (ms), simulated	45	120	210	420	680

The source material describes the scaling as “linear with log(n)”. We examined that description against the five reported points, because it determines whether the logarithmic bound of Section 4.5 can be said to be supported. We fitted three candidate laws by least squares (computed by us from the five values; no other data were used).

Model	Fit	Quality	Observation
Linear in ln n	$T \approx 156 \cdot \ln n - 409$	$R^2 = 0.85$	Predicts –50 ms at n = 10; local slope per unit of ln n rises from 47 to 375 ms across the range, so the data are not linear in ln n.
Linear in n	$T \approx 1.28 \cdot n + 62$	$R^2 = 0.99$	Good fit overall, but local slope falls from 1.9 to 1.0 ms per node.
Power law	$T \approx 8.4 \cdot n^{0.70}$	$R^2 = 0.997$ (log-log)	Best of the three; exponent between logarithmic and linear.

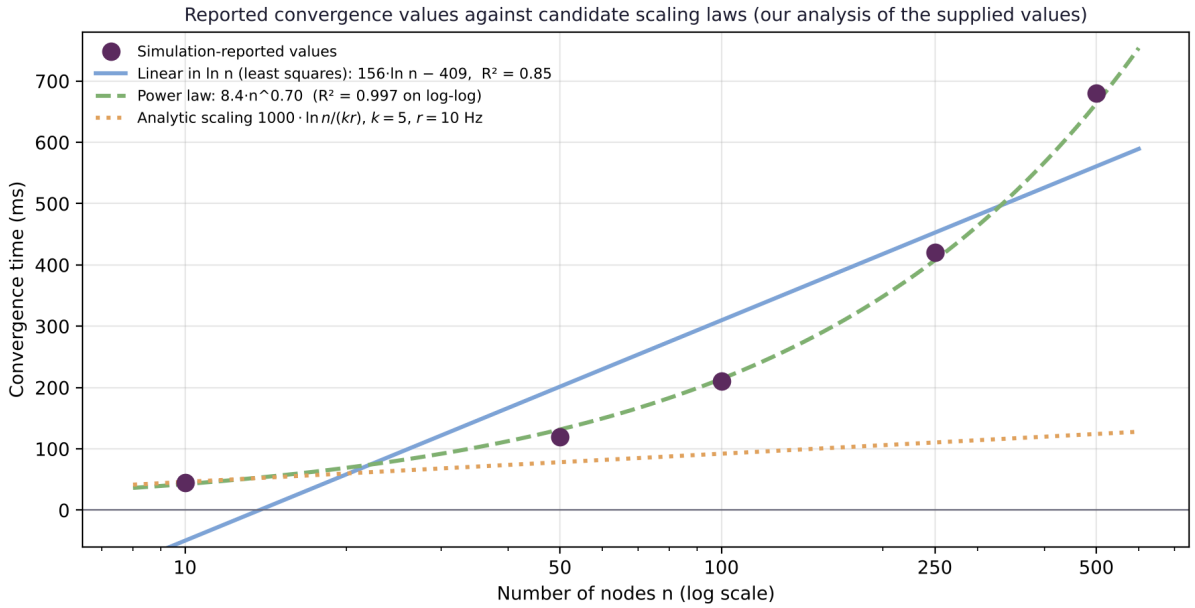


Figure 9 (Fig6_Scaling_Analysis.png; supplementary). The five simulation-reported convergence times with three fitted laws and the analytic scaling of Equation (4) for k = 5, r = 10 Hz. Fits and the analytic curve are computed by us; only the five points are simulation-reported.

The result of this analysis is a correction to how the data should be described. **The reported values are not consistent with scaling that is linear in log n**; they are better described by a sub-linear power law with an exponent near 0.7, and the gap between the analytic estimate of Equation (4) and the reported values grows from 1.0× at n = 10 to 5.5× at n = 500 (Table 4.2). With only five points and two or three fitted parameters, the exponent is imprecise, and we do not claim that the true scaling law is a power law. We also do not know the number of runs or their variance. What the data do support is the narrower statement that, in this simulation, convergence at 250 nodes was reported as 420 ms, within the 450 ms

reference, and that latency increased by about a factor of 15 while the node count increased by a factor of 50.

Several explanations for the super-logarithmic growth are conceivable and none has been tested: validation work that grows with the number of deltas or peers, fixed fanout over partitioned topologies whose diameter increases with n, contention in the simulated message queues, and the cost of repeated digest comparison. We list them as hypotheses for future work (Q22). The 99% threshold itself matters: the last 1% of nodes, in the tail of the epidemic, can dominate the time to full convergence.

5.3 Metric 2 — Conflict Resolution Overhead

Definition. CPU and memory overhead during high-concurrency conflicting edits.

Simulation-reported results. The CRDT-based join eliminated 100% of blocking deadlocks relative to a traditional lock-based version control baseline; CPU overhead was below 5% during peak conflict; memory overhead was below 10% above baseline.

Interpretation. The deadlock result should be understood as a property of the design more than as an empirical finding: a join over a semilattice does not acquire locks and cannot block on one, so the absence of lock deadlocks follows from the architecture, and the simulation confirms that the simulated implementation did not introduce locking. The comparison with a lock-based system presupposes that the baseline deadlocked at a non-zero rate under the workload, which the record does not state. The CPU and memory figures are expressed relative to a ‘baseline’ whose definition is not given, and the figures are upper bounds (‘below’) rather than point estimates. They should be read as indicating that no large overhead was observed in the simulation, not as a cost model.

Conflict *resolution* in AetherMesh does not remove semantic conflicts; it makes the merge deterministic. Conflicts flagged by the compatibility matrix (Table 3.1) remain conflicts in the program sense, and the simulation does not assess how many such conflicts arise or how they are resolved.

5.4 Metric 3 — Fault Resilience

Definition. Network resilience under Byzantine injection of malformed deltas.

Simulation-reported results. The Pillar 3 verification layer blocked 100% of invalid state transitions; throughput overhead was below 2%; no valid deltas were rejected (0% false positives).

Interpretation. Malformed deltas, that is, deltas with bad signatures, broken structure, or schema violations, are exactly the class that the checks of Section 3.3 are designed to detect, so rejection of all of them is the expected outcome. The result is a check that the simulated pipeline is wired correctly, not evidence of resilience against adversaries who produce well-formed deltas. The record does not state how many deltas were injected. If N malformed deltas were injected and none was admitted, the one-sided 95% upper confidence bound on the admission probability is approximately 3/N (the ‘rule of three’); for example N = 1,000 would bound it near 0.3%, and N = 100 near 3%. Without N, the claim of ‘100%’ has no denominator, and we recommend that any future report state it.

5.5 Summary Table

#	Metric	Simulation-reported result	Reading
1	State convergence latency	420 ms at 250 nodes (below 450 ms); 45–680 ms over 10–500 nodes	Reported; scaling is not log-linear (Section 5.2)
2	Conflict-resolution overhead	No blocking deadlocks; CPU below 5%; memory below 10%	Deadlock-freedom follows from design; overhead baseline undefined
3	Fault resilience	All injected malformed deltas blocked; 0% false positives; throughput overhead below 2%	Pipeline check for predicate-detectable faults; number injected unreported

5.6 Limitations

- **Simulated data.** No real-world deployment has been evaluated, and no result in this section is a measurement of a deployed system.
- **Scale.** The testbed covers 10–500 nodes; behaviour at thousands of nodes is unknown, and the observed super-logarithmic growth suggests that extrapolation would be unsafe.
- **Simplified network model.** No real latency, packet loss, churn, NAT, or adversarial network conditions.
- **Byzantine model.** Limited to malformed deltas. Colluding adversaries, equivocation, replay, withholding, flooding, and Sybil identities were not evaluated.
- **Missing statistics.** Number of runs, variance, and confidence intervals are not reported; simulator source and seeds are not included.
- **No ablation.** The contribution of each pillar to latency and overhead was not isolated, and no comparison with a gossip-only or a consensus-based design was reported.

Limitations of this section. This section reports what was supplied and analyses it; it does not add new experiments. Our scaling analysis uses five data points and is descriptive only.

6. Discussion

6.1 What Decentralization Adds

The architecture offers four properties that a centralized orchestrator does not, each conditional on the assumptions of Section 4. *Availability under partition:* every node can accept and merge deltas locally while disconnected, and the lattice merge reconciles the divergence afterwards. *No single point of failure:* no node is distinguished, so the loss of any one does not stop the others. *Parallelism without waiting:* agents never wait for a lock or for a sequencer. *Local autonomy:* each node applies its own admission pipeline and can continue operating without consulting a central authority.

Each property has a price. Availability under partition means that replicas can be observably different for a while, so any consumer that reads a replica must tolerate stale or divergent views. Removing the sequencer removes a global order, so operations that need one (Section 6.3) are not supported. Local autonomy shifts the burden of trust establishment to each node. The architecture trades coordination cost for these burdens; whether the trade is favourable depends on the workload, which this paper has not measured.

6.2 The CRDT–Gossip–BFT Trilemma

The three pillars are not independent, and improving one can degrade another. We describe three tensions as an engineering framing, not as a proven impossibility. The analogy to the CAP trade-off (Gilbert and Lynch, 2002), in which a system must give up consistency or availability during partitions, is loose: AetherMesh chooses availability and eventual consistency, and the tensions below concern what else is lost.

Tension	Between	Why they pull apart	Design response in this paper
Merge expressiveness vs. coordination-freedom	CRDT and semantic validity	Rich semantic merge rules tend to need coordination; coordination-free merges are semantically coarse	Merge the delta set coarsely; push semantic checks to admission and to higher layers
Dissemination speed vs. verification cost	Gossip and BFT	Each hop that verifies before forwarding adds latency; forwarding unverified deltas spreads invalid ones	Verify before join; forwarding policy is open (D31); report overhead as simulated (below 2%)
Filter strength vs. convergence	BFT and CRDT	Stronger predicates tend to depend on more state, which risks verdict divergence and thus non-convergence	Admission determinism requirement (A6); local policies only delay, never reject

6.3 When Not to Use AetherMesh

- **Small, co-located teams or few agents.** A conventional central repository has lower complexity and, at this scale, probably adequate performance.
- **Strong consistency requirements.** Unique-name allocation, exactly-once external effects, release gating, and any operation that needs a global order are not expressible as join-semilattice updates.
- **Validity must hold before visibility.** If a merged program must be known to be correct before any agent can observe it, a gate before merge is required; AetherMesh admits deltas on syntactic and compatibility grounds only.
- **Adversarial settings beyond the stated threat model.** Where colluding or well-funded adversaries are expected, a lightweight filter is insufficient, and consensus-based or hardware-attested designs should be considered.
- **Constrained nodes.** Because the delta set grows without compaction (Q9), nodes with small storage budgets may not be able to hold replicas.

6.4 Relation to Prior Papers in the Series

The four layers can be composed, but the composition has not been built or evaluated; the following describes the intended relationships and their risks.

Prior layer	Relationship to AetherMesh	Open risk
AetherGit (storage)	Its append-only, parent-pointer changeset log supplies the delta store; replication of the log is what gossip performs.	Mapping between its changeset identifiers and AetherMesh delta identifiers is unspecified.
AetherFlow (integration)	Its post-merge verification and remediation can run per node over the materialized view; a remediation is itself a delta and travels by gossip.	Different nodes may reach different verification verdicts at different times; verdicts must not feed back into admission unless deterministic over the causal closure.
AetherProof (verification)	Proof evidence may be attached to deltas as metadata and consulted by semantic predicates.	Evidence validity is conditional on assumptions and on artifact stability in that paper's own terms; consulting evidence in admission may violate A6 if evidence is evaluated non-deterministically.

DOI: https://doi.org/10.5281/zenodo.24038038

Limitations of this section. The discussion is qualitative. The trilemma is a framing and is not proven. The cross-layer compositions have not been built, and the risks listed may not be exhaustive.

7. Threats and Limitations

7.1 Threat Taxonomy

We organize threats into six classes. Fifteen threats, T1–T15, are catalogued with mitigations and residual risk in Appendix D; this section discusses each class.

Class	Question	Threats
Soundness	Can the architecture admit something it should not, or converge to something wrong?	T1–T3
Completeness	Can it fail to admit or detect what it should?	T4–T5
Trust	Do the trust assumptions hold?	T6–T8
Process	Does it behave under load, partition, and growth?	T9–T11
Toolchain	Do the simulator and implementation components faithfully represent the design?	T12–T13
Human	Do operators and readers use it correctly?	T14–T15

7.2 Soundness Threats

T1, admission nondeterminism. A predicate that depends on local state can cause honest nodes to admit different sets and diverge permanently. The determinism requirement (A6) addresses this by design, but nothing enforces it. **T2, the semantic-merge gap.** Individually valid deltas can combine into an invalid program; the compatibility matrix is coarse and the lattice does not address it. **T3, proof-sketch gaps.** Theorems 1–3 are sketches; a subtle error, for example in the treatment of fairness or deferral, may remain.

7.3 Completeness Threats

T4, harmful but well-formed deltas. A registered agent that is compromised or misaligned can submit deltas that satisfy every predicate and still do harm. The architecture has no defence against this beyond what higher layers provide. **T5, false rejection.** An over-strict schema or matrix, or schema version skew between nodes, can reject valid deltas; because verdicts are deterministic, the rejection is consistent across nodes but may be wrong everywhere. The simulation reported no false rejections, but it did not test schema evolution.

7.4 Trust Threats

T6, key registry. Provenance is only as good as the key registry. Bootstrap, rotation, and revocation are not specified, and compromise of a key allows well-formed forged deltas under that identity. **T7, collusion and Sybil identities.** The $f < n/3$ condition is meaningful only if identities are scarce; a Sybil attacker (Douceur, 2002) can exceed the fraction by creating identities. **T8, eclipse and peer-sampling bias.** An adversary that dominates a node's peer sample can isolate it, delay dissemination, or feed it selected deltas; the uniform selection that Theorem 2 assumes is exactly what such an attack subverts.

7.5 Process Threats

T9, resource exhaustion. Flooding with valid-looking deltas, oversized digests, or deltas with unsatisfiable parent pointers can exhaust CPU, bandwidth, or the deferral buffer. Per-origin rate limits mitigate this; as local delivery policies, they delay rather than reject. **T10, history growth.** The delta set only grows; compaction must preserve join semantics. **T11, long partitions.** After a long partition, healing may require a large merge, and the admission cost of the backlog may be considerable and was not simulated.

7.6 Toolchain Threats

T12, simulator fidelity. The simulator uses a simplified network model; results may not transfer. **T13, implementation faults.** Cryptographic library bugs, serialization differences, and schema version skew between nodes can all break the determinism that the theorems assume.

7.7 Human Threats

T14, misconfiguration. Operators set fanout, period, matrix entries, and trust anchors; poor choices can degrade convergence or admit unwanted deltas. **T15, overreliance on simulated results.** Readers or downstream users may treat simulated figures as evidence of deployed performance. This paper's labelling is intended to prevent that; it cannot enforce it.

7.8 Threats to Validity

Type	Threat	Status
Internal	Unknown simulator parameters, runs, and seeds; possible simulator artifacts; no ablation of pillars	Open
External	Simulated network, 10–500 nodes, malformed-delta-only adversary; no real code workloads	Open; results do not generalize without further evaluation
Construct	‘Convergence’ measured as 99% merged, not 100%; ‘invalid’ means predicate-detectable; ‘deadlock elimination’ is partly by design	Stated in Section 5
Conclusion	Five data points for scaling; no confidence intervals; proofs are sketches	Stated in Sections 4 and 5

7.9 Limitations

Architectural limitations. The architecture provides eventual, not strong, consistency; it offers no global order; it filters only predicate-detectable invalid deltas; it assumes a trustworthy key registry; and it grows without compaction. **Design limitations.** The compatibility matrix, digest form, fanout policy, forwarding policy, and compaction are underspecified; the determinism requirement is not enforced mechanically. **Scope limitations.** Dynamic membership, access control beyond key registration, privacy of deltas, and integration with the other layers are not treated.

Limitations of this section. The threat catalogue is not exhaustive and its mitigations are design intentions that have not been tested. Residual risk entries in Appendix D are our qualitative judgements, not measurements.

8. Conclusion and Future Work

8.1 Summary

This paper specified AetherMesh, a decentralized architecture that combines CRDT merging of code and metadata deltas, epidemic gossip dissemination, and local Byzantine admission, as the decentralization layer of the Aether stack. It stated three theorems with proof sketches under explicit assumptions, identified an admission determinism requirement necessary for the combination to converge, and reported a simulated evaluation on 10–500 nodes. In that simulation, convergence at 250 nodes was reported as 420 ms, no blocking deadlocks occurred, and all injected malformed deltas were blocked. Our analysis of the reported scaling showed that the values are not consistent with logarithmic growth, and that the analytic latency estimate diverges from them as the network grows.

The evidence supports a modest conclusion: the architecture is internally coherent, its central convergence argument is plausible under stated assumptions, and a simplified simulation did not contradict it. It does not support claims about deployed performance, about adversaries beyond malformed deltas, or about the validity of merged programs.

8.2 Future Work

Direction	Description	Addresses
Real-world evaluation	Deploy across geo-distributed machines; measure convergence under real latency, loss, and churn; report parameters, runs, and variance	T12, Q21–Q25
Mechanized proofs	Model channels, fairness, deferral, and admission in a proof assistant or model checker; prove Theorem 1 over that model	T3, Q1–Q3
Extended Byzantine models	Colluding, equivocating, withholding, and Sybil adversaries; eclipse attacks on peer sampling	T4, T7, T8
Trusted execution	Hardware attestation of agent identity and state, reducing reliance on a key registry alone	T6, Q32
Finer-grained merging	Tree and AST CRDTs that merge below the granularity of a delta; evaluate semantic-merge gap	T2, Q5
Compaction and membership	Safe garbage collection of the delta set; dynamic membership and key rotation	T10, Q9, Q31
Layer integration	Study the composition with the storage, integration, and verification layers, including verdict divergence	Section 6.4

The simulation results here are an invitation to such evaluation, not a substitute for it.

Limitations of this section. Future-work items are directions, not commitments, and their relative importance is our judgement.

DOI: https://doi.org/10.5281/zenodo.24038038

Appendix A — Notation

Symbol	Meaning
$A = \{a_1, \ldots, a_n\}$	Set of n autonomous agents (nodes)
f	Number of Byzantine agents
$R_a^{(t)}$	Local replica state of agent a at logical time t
$(S, \sqcup, \bot)$	Bounded join-semilattice: state space, join, bottom element
$\sqcup$, $\bigsqcup$	Binary join; iterated join over a set
$\sqsubseteq$	Lattice order: $s \sqsubseteq t$ iff $s \sqcup t = t$
$\top$	Top element (join of all elements of a finite lattice)
D	Universe of deltas that may ever be created (finite by A3)
Δ_i	A CRDT delta: signed, content-addressed record of one change with parent pointers
M_a	Set of incoming (admitted) mutation payloads at agent a during (t, t+1)
M*	Set of all deltas admitted by some non-faulty node
$s_{reconciled}$	Reconciled state $s_i \sqcup s_j$ (Definition 2)
$P_{uninfected}(t)$	Probability that a node is not yet reached by step t
β	Gossip transmission rate constant (per step)
p	Per-step infection probability in the constant-hazard model; $\beta = -\ln(1 - p)$
k	Gossip fanout (peers contacted per round)
r	Gossip round frequency (rounds per second)
$T_{convergence}$	Heuristic convergence time estimate, Equation (4)
m	Size of an attestation committee (Theorem 3, part b)
$D(p \parallel q)$	Kullback–Leibler divergence between Bernoulli(p) and Bernoulli(q)
C	Semantic compatibility matrix over operation kinds
A1–A6	Assumptions of Section 4.0
T1–T15, Q1–Q50, D1–D52	Threats, open questions, design decisions (Appendices D, B, C). Note: T1–T3 as threats are distinct from Theorems 1–3.

Appendix B — Open Questions Q1–Q50

The following questions are open. Items are grouped by theme; none is claimed to be answered by this paper.

Formal analysis

ID	Question
Q1	Can Theorem 1 be mechanized over an explicit model of fair-lossy channels, gossip fairness, deferral, and admission?
Q2	What is the weakest fairness assumption on peer selection that still yields eventual consistency?
Q3	Can Theorem 2 be replaced by a full two-phase analysis (growth then decay) for the fanout and digest forms actually used?
Q4	Can the admission determinism requirement (A6) be checked mechanically for a given predicate set, or enforced by construction?
Q5	What granularity of tree or AST CRDT merges code below the granularity of a delta without losing convergence?
Q6	How often does the semantic-merge gap occur in realistic agent workloads, and how much of it does a compatibility matrix catch?
Q7	What are the correct semantics when verdicts of higher layers (verification, remediation) arrive after admission?
Q8	What is a formal semantics for deferral that guarantees bounded buffering without permanent rejection of valid deltas?

State and storage

ID	Question
Q9	How can the delta set be compacted safely while preserving join semantics and auditability?
Q10	What snapshot mechanism lets a new node bootstrap without replaying the full history?
Q11	Which materialization linearization rule minimizes spurious conflicts while remaining deterministic?
Q12	Which metadata lattices are sufficient for ownership, status, and dependency annotations?
Q13	How should deltas be sized: per edit, per task, per agent step?
Q14	Can causal stability be detected in a decentralized way to enable garbage collection?
Q15	How does pruning history interact with provenance and forensic needs?
Q16	What are storage budgets per node as a function of agent count and change rate?

Gossip and performance

ID	Question
Q17	Which digest form (version vector, Merkle tree, filter) is best as a function of agent count and churn?
Q18	What fanout and period minimize convergence time under a bandwidth budget?
Q19	When is push, pull, or push-pull preferable for code deltas?
Q20	How much does topology-aware peer selection improve latency, and what does it cost in analytic guarantees?
Q21	What convergence times occur under real network latency, loss, and jitter?
Q22	Why do the reported simulated latencies grow faster than logarithmically, and which of the hypothesized causes dominates?
Q23	How does churn (nodes joining and leaving) affect convergence?
Q24	What is the cost of healing after long partitions as a function of backlog size?
Q25	What are the tail and variance of convergence time, and how does the 99% criterion relate to full convergence?

Byzantine tolerance and trust

ID	Question
Q26	How expressive can admission predicates be while remaining deterministic over the causal closure?
Q27	How should compatibility matrices be authored, versioned, and validated?
Q28	What quarantine semantics are appropriate for deltas flagged but not rejected?
Q29	Can local reputation signals be used without breaking determinism?
Q30	How should equivocation (conflicting deltas with the same origin and parent) be detected and handled?
Q31	How should key rotation, revocation, and dynamic membership work without a central authority?
Q32	What role can trusted execution environments play in attesting agent identity and state?
Q33	What Sybil-resistance mechanism suits agent populations without a central registry?
Q34	What peer-sampling defences resist eclipse attacks without weakening uniformity?
Q35	Should nodes forward deltas before verifying them, and what are the latency and safety consequences?

Integration with other layers

ID	Question
Q36	How should storage-layer changeset identifiers map to delta identifiers?
Q37	How should post-merge verification verdicts be made consistent across nodes?
Q38	How should proof evidence be attached to deltas and consulted without violating determinism?
Q39	How should remediation deltas be authored and authorized in a decentralized setting?
Q40	What human-review role exists when no central reviewer does?
Q41	What audit trail does the architecture need and who can read it?
Q42	What privacy protections should apply to deltas that traverse untrusted peers?

Evaluation method

ID	Question
Q43	What reproducibility package (parameters, seeds, source, traces) should accompany simulated results?
Q44	How should the contribution of each pillar be isolated by ablation?
Q45	What baselines (gossip-only, consensus-based, centralized) are fair comparators?
Q46	What realistic code-change workloads should drive evaluation?
Q47	What adversarial benchmark would test colluding and equivocating agents?
Q48	How does the architecture behave at thousands of nodes?
Q49	What are the energy and monetary costs of verification and gossip per delta?
Q50	How do operators configure and misconfigure the system, and what human-factors evidence exists?

Appendix C — Design Decisions D1–D52

Each decision records a choice made in this specification and a brief rationale. Decisions are design choices, not validated optima.

ID	Decision	Rationale
D1	Four layers are methodologically independent	Allows each paper to be read and evaluated alone
D2	Require only an append-only, parent-pointer log from storage	Minimal coupling
D3	Require only a message-based protocol from integration	Minimal coupling
D4	Three pillars: CRDT, gossip, lightweight BFT	Separates merge, dissemination, and trust
D5	No central coordinator	Removes single point of failure
D6	No global locks	Removes lock deadlocks by construction
D7	Eventual rather than strong consistency	Compatible with coordination-free operation
D8	Availability over consistency under partition	Agents continue working when disconnected
D9	Replica state is the set of admitted deltas	Simplest lattice; convergence argument independent of file structure
D10	Set union is the join	Commutative, associative, idempotent
D11	Materialization is a deterministic function of the delta set	Equal sets give equal codebases
D12	Linearize by topological order with identifier tie-break	Deterministic and parent-respecting
D13	Metadata modeled as a product lattice	Product of semilattices is a semilattice
D14	Observed-remove sets for membership-style metadata	Standard CRDT with add-wins semantics
D15	Last-writer-wins registers with agent-id tie-break	Deterministic single-valued metadata
D16	Digest form is a parameter: version vector, Merkle tree, or filter	Trade-off depends on population and churn
D17	Delta-state replication by default	Bandwidth near operation-based, algebra of state-based
D18	State-based fallback for large divergence and joining nodes	Cheaper than long incremental repair
D19	Operation-based style not used on the wire	Avoids causal-delivery requirements
D20	Deltas are content-addressed	Tamper-evident; cycle-free
D21	Parent identifiers are inside the hashed content	Binds history into identity
D22	Uniform random peer selection	Matches assumption A2
D23	Partial-view peer sampling	Avoids full membership lists
D24	Fanout k is a configuration parameter	Controls β
D25	Gossip period 1/r is a configuration parameter	Controls β
D26	Push-pull exchanges by default	Faster late-phase dissemination
D27	Anti-entropy provides repair	Implicit retransmission supports A1
D28	Digests re-offered each round	Realizes fair-lossy delivery in practice
D29	Fair-lossy channels, not bounded delay, for Theorem 1	Weaker, more realistic assumption
D30	Verify before join	Invalid deltas never enter the lattice
D31	Forwarding policy: verify before forward (default; open question)	Limits spread of invalid deltas at some latency cost
D32	Ed25519-class signatures are representative	Fast, widely reviewed
D33	Signature covers the content hash	Covers body and parents
D34	Local key registry per node	No central authority
D35	Missing parents defer rather than reject	Gossip may deliver children first
D36	Deferral buffer is bounded in size and time	Resource-exhaustion defence

ID	Decision	Rationale
D37	Schema conformance check	Cheap structural filter
D38	Semantic compatibility matrix over operation kinds	Cheap local semantic filter
D39	Matrix verdicts: ok, resolve, conflict	Small, decidable vocabulary
D40	Quarantine is an optional outcome for conflicts	Avoids silent loss
D41	Admission determinism over the causal closure (A6)	Required for convergence of admitted sets
D42	Local policies may delay but never permanently reject	Preserves eventual admitted-set equality
D43	Per-origin rate limits	Flooding defence
D44	Size and fan-in bounds on deltas	Resource bounds
D45	Rejections are local; no global blacklist by default	Avoids a new coordination point
D46	Theorem 3 scoped to predicate-detectable invalid deltas	Honest scope of local filtering
D47	Quorum only for decisions that cannot be made locally	Confines the n/3 bound to where it is relevant
D48	Committee sampling is uniform	Enables Chernoff-style bound
D49	Theorems are labeled sketches, not machine-checked	Avoids overclaiming
D50	Evaluation is simulation-only and labeled as such	No deployment claimed
D51	Report unknown parameters explicitly	Supports critical reading
D52	Conservative language; no promotional claims	Matches evidentiary status

Appendix D — Threats T1–T15

ID	Class	Threat	Mitigation (design intent)	Residual risk (our judgement)
T1	Soundness	Admission nondeterminism causes divergent admitted sets	A6; delay-only local policies	Medium: not mechanically enforced
T2	Soundness	Semantic-merge gap: valid deltas combine into invalid program	Compatibility matrix; higher-layer verification	High: matrix is coarse
T3	Soundness	Proof sketches contain errors	Mechanization (future); explicit assumptions	Medium
T4	Completeness	Well-formed but harmful deltas admitted	Higher layers; review; provenance for attribution	High: no defence in this layer
T5	Completeness	False rejection from strict schema or version skew	Versioned schemas; quarantine option	Medium
T6	Trust	Key registry bootstrap or compromise	Rotation and revocation (unspecified); TEEs (future)	High
T7	Trust	Collusion or Sybil identities exceed n/3	Scarce identities; quorum committee sampling	High under open membership
T8	Trust	Eclipse or peer-sampling bias	Partial-view refresh; diverse peers	Medium to high
T9	Process	Flooding, oversized digests, deferral exhaustion	Rate limits; size bounds; bounded deferral	Medium
T10	Process	Unbounded history growth; unsafe compaction	Compaction research (Q9)	Medium
T11	Process	Costly healing after long partitions	State-based fallback; rate-limited admission	Unknown: not simulated
T12	Toolchain	Simulator infidelity to real networks	Real-world evaluation (future)	High until evaluated
T13	Toolchain	Crypto-library, serialization, or schema skew faults	Canonical encoding; versioning; testing	Medium
T14	Human	Operator misconfiguration of parameters, matrix, or trust anchors	Defaults; review; documentation	Medium
T15	Human	Reliance on simulated results as deployed evidence	Labeling throughout; Research Status Statement	Medium: cannot be enforced

Residual-risk entries are qualitative judgements by the author, offered to prioritize future work; they are not measurements.

References

Almeida, P. S., Shoker, A., & Baquero, C. (2018). Delta state replicated data types. *Journal of Parallel and Distributed Computing, 111*, 162–173.

Bailis, P., & Ghodsi, A. (2013). Eventual consistency today: Limitations, extensions, and beyond. *Communications of the ACM, 56*(5), 55–63.

Baladi, S. (2026a). *AetherGit: Scalable Source Control for Autonomous AI Agents*. Zenodo. https://doi.org/10.5281/zenodo.23966896

Baladi, S. (2026b). *AetherFlow: Automated Post-Merge Verification and Dynamic Conflict Resolution in Multi-Agent Programming Environments*. Zenodo. https://doi.org/10.5281/zenodo.23979129

Baladi, S. (2026c). *AetherProof: Autonomous Formal Verification and Invariant Synthesis for Multi-Agent Software Ecosystems*. Zenodo. https://doi.org/10.5281/zenodo.24013124

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering, 2*(2), 77–89.

Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. In *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (pp. 173–186).

Das, A., Gupta, I., & Motivala, A. (2002). SWIM: Scalable weakly-consistent infection-style process group membership protocol. In *Proc. International Conference on Dependable Systems and Networks (DSN)* (pp. 303–312).

DeCandia, G., Hastorun, D., Jampani, M., et al. (2007). Dynamo: Amazon's highly available key-value store. In *Proc. 21st ACM SOSP* (pp. 205–220).

Demers, A., Greene, D., Hauser, C., et al. (1987). Epidemic algorithms for replicated database maintenance. In *Proc. 6th ACM Symposium on Principles of Distributed Computing (PODC)* (pp. 1–12).

Douceur, J. R. (2002). The Sybil attack. In *Peer-to-Peer Systems (IPTPS 2002)*, LNCS 2429 (pp. 251–260). Springer.

Fischer, M. J., Lynch, N. A., & Paterson, M. S. (1985). Impossibility of distributed consensus with one faulty process. *Journal of the ACM, 32*(2), 374–382.

Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News, 33*(2), 51–59.

Hong, S., Zhuge, M., Chen, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proc. ICLR 2024*.

Jelasity, M., Montresor, A., & Babaoglu, O. (2005). Gossip-based aggregation in large dynamic networks. *ACM Transactions on Computer Systems, 23*(3), 219–252.

Jimenez, C. E., Yang, J., Wettig, A., et al. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR 2024*.

Karp, R., Schindelhauer, C., Shenker, S., & Vöcking, B. (2000). Randomized rumor spreading. In *Proc. 41st IEEE Symposium on Foundations of Computer Science (FOCS)* (pp. 565–574).

Kermarrec, A.-M., & van Steen, M. (2007). Gossiping in distributed systems. *ACM SIGOPS Operating Systems Review, 41*(5), 2–7.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Kleppmann, M., & Beresford, A. R. (2017). A conflict-free replicated JSON datatype. *IEEE Transactions on Parallel and Distributed Systems, 28*(10), 2733–2746.

Kleppmann, M., & Howard, H. (2020). Byzantine eventual consistency and the fundamental limits of peer-to-peer databases. arXiv:2012.00472.

Kleppmann, M., Mulligan, D. P., Gomes, V. B. F., & Beresford, A. R. (2022). A highly-available move operation for replicated trees. *IEEE Transactions on Parallel and Distributed Systems, 33*(7), 1711–1724.

Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems, 4*(3), 382–401.

Merkle, R. C. (1988). A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, LNCS 293 (pp. 369–378). Springer.

Pittel, B. (1987). On spreading a rumor. *SIAM Journal on Applied Mathematics, 47*(1), 213–223.

Qian, C., Liu, W., Liu, H., et al. (2024). ChatDev: Communicative agents for software development. In *Proc. ACL 2024*.

Roh, H.-G., Jeon, M., Kim, J.-S., & Lee, J. (2011). Replicated abstract data types: Building blocks for collaborative applications. *Journal of Parallel and Distributed Computing, 71*(3), 354–368.

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011a). Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, LNCS 6976 (pp. 386–400). Springer.

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011b). *A comprehensive study of convergent and commutative replicated data types* (Research Report RR-7506). INRIA.

Vogels, W. (2009). Eventually consistent. *Communications of the ACM, 52*(1), 40–44.

Vukoli■, M. (2015). The quest for scalable blockchain fabric: Proof-of-work vs. BFT replication. In *Open Problems in Network Security (iNetSec 2015)*, LNCS 9591 (pp. 112–125). Springer.

Weiss, S., Urso, P., & Molli, P. (2009). Logoot: A scalable optimistic replication algorithm for collaborative editing on P2P networks. In *Proc. 29th IEEE ICDCS* (pp. 404–412).

Note: the reference list was assembled from the author's knowledge of the literature, not by live bibliographic lookup. Bibliographic details (volumes, pages, venues) should be verified before submission.

The three companion papers (Baladi, 2026a, 2026b, 2026c) are cited from their Zenodo records.

Research Status Statement

Item	Status
Document type	Research article: formal architecture proposal with simulated evaluation
Version / record	1.0.0, November 2026 · AETHERMESH-01 · DOI 10.5281/zenodo.24038038
Implementation	None claimed or evaluated in this paper
Real-world deployment	None; no deployed system has been benchmarked
Theorems 1–3	Stated with proof sketches; not machine-checked; conditional on assumptions A1–A6
Numerical results	All from a simulated P2P testbed (10–500 nodes) with a simplified network model; simulator source, seeds, and raw traces not included
Byzantine evaluation	Limited to malformed deltas; no colluding, equivocating, or Sybil adversary evaluated
Third-party claims	As reported by cited sources; not independently verified
Review status	Not peer reviewed

Reconciliation notes

In preparing this record, the following departures from the supplied draft material were made, so that readers can see what was changed and why.

1 **Theorem status.** Draft summaries marked the theorems “Proven”. Following the paper's own stated requirement, they are labeled as sketches, not machine-checked.

2 **Delay assumption.** “Bounded message delay” in Theorem 1 was replaced for the qualitative claim by fair-lossy channels (A1); boundedness is relevant only to time bounds.

3 **Admission determinism.** Assumption A6 and Section 3.3.4 were added, because without it local verification can prevent convergence.

4 **Theorem 3.** The statement is preserved; its scope is clarified to predicate-detectable invalid deltas, and the n/3 bound is placed in the quorum-based part with a Chernoff–Hoeffding bound that we derived.

5 **Latency bound.** The draft stated that the 110 ms estimate “matches” the experimental result. Table 4.2 shows it does not match across sizes; the text now says so.

6 **Scaling claim.** “Linear with log(n)” is not supported by the five reported points (Section 5.2); the text reports our fits.

7 **Deadlock elimination and 100% blocking.** Both are noted as following largely from design and from the class of faults injected; the denominators were not supplied.

8 **Figures.** The five supplied figures were redrawn (the originals showed a placeholder DOI and a missing glyph, used the step and check numbering inconsistently, and carried plotting artefacts). Figures 6–9 are supplementary and have no supplied counterpart. Figure and filename mapping is given in each caption.

9 **Conclusion wording.** Phrases such as “successfully bridges” and “demonstrates … achievable” were softened to match the evidence.

10 **Format.** The supplied prompt requested Markdown; this record is a PDF, as requested separately.

11 **References.** Assembled from knowledge; verification before submission is recommended.

Author information. Samir Baladi, Interdisciplinary AI Researcher, Ronin Institute / Rite of Renaissance. Email: gitdeeper@gmail.com. ORCID: 0009-0003-8903-0029. **Citation:** Baladi, S. (2026). AetherMesh: Decentralized Peer-to-Peer Agent Synchronization and State CRDTs for Autonomous Multi-Agent Systems. Zenodo. https://doi.org/10.5281/zenodo.24038038